

Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

{ FACHBEREICH
INFORMATIK }

Programmierung 1

Methoden

WS 2026/27

Bachelor Cyber Security & Privacy (B.Sc.)

André Kless, M.Sc.

Algorithmen wiederverwenden

Unser Programm braucht den ggT an mehreren Stellen:

48 und 18

ggT: 6

35 und 14

ggT: 7

81 und 27

ggT: 27

Müssen wir denselben Algorithmus dafür immer wieder hinschreiben?

Unser Ziel: Den Code einmal schreiben und
an jeder benötigten Programmstelle wiederverwenden.

Variante 1: Den Code dreimal hinschreiben

Die Abschnitte stehen nacheinander in `main`. Die Startwerte sind positiv.

1. Berechnung

```
int a = 48;
int b = 18;
while (a != b) {
    if (a > b) {
        a = a - b;
    } else {
        b = b - a;
    }
}
IO.println(a); // 6
```

2. Berechnung

```
a = 35;
b = 14;
while (a != b) {
    if (a > b) {
        a = a - b;
    } else {
        b = b - a;
    }
}
IO.println(a); // 7
```

3. Berechnung

```
a = 81;
b = 27;
while (a != b) {
    if (a > b) {
        a = a - b;
    } else {
        b = b - a;
    }
}
IO.println(a); // 27
```

Nachteil: Drei Kopien des Algorithmus. Bei einer Änderung müssen wir alle anpassen.

Variante 2: Die Startwerte einlesen

Wir lesen positive Startwerte ein. Der ggT-Algorithmus bleibt im Programm.

```
int a = Integer.parseInt(IO.readln("a: "));
int b = Integer.parseInt(IO.readln("b: "));
while (a != b) {
    if (a > b) {
        a = a - b;
    } else {
        b = b - a;
    }
}
IO.println(a);
```

Vorteil

Andere Zahlen berechnen, ohne den Quelltext zu ändern.

Nachteil

Braucht das Programm den ggT an anderer Stelle, müssten wir dort den Algorithmus erneut hinschreiben.

Variante 3: Eine eigene Methode aufrufen

Der ggT-Algorithmus steht einmal in `ggt`. An jeder benötigten Stelle genügt ein Aufruf.

Drei Aufrufe

```
void main() {
    IO.println(ggt(48, 18)); // 6
    IO.println(ggt(35, 14)); // 7
    IO.println(ggt(81, 27)); // 27
}
```

Auch eingelesene Werte können wir übergeben: `ggt(a, b)`.

Eine Definition

```
int ggt(int a, int b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}
```

Vorteil: Denselben Algorithmus mehrfach nutzen und nur an einer Stelle ändern.

DRY-Prinzip: Don't Repeat Yourself

„Wiederhole dich nicht.“

Dieselbe fachliche Regel soll nur an einer maßgeblichen Stelle festgelegt sein.
Für wiederverwendbare Berechnungen schreiben wir eine Methode.

Beim ggT

Den Algorithmus einmal in `ggt` definieren.
Jede benötigte ggT-Berechnung erfolgt über einen Aufruf dieser Methode.

Warum das hilft

Eine Korrektur gilt für alle Aufrufe.
Wir müssen keine Kopien einzeln anpassen und aktuell halten.

Prüffrage: Muss ich dieselbe fachliche Regel an mehreren Stellen ändern?
Dann kann eine gemeinsame Methode diese Wiederholung vermeiden.

The Pragmatic Programmer: DRY-Prinzip (Originaltext)

Die Methode als Blackbox



Argumente hinein

Rückgabewert heraus

Eine Methode ist ein benannter Programmabschnitt, den wir bei Bedarf aufrufen.
Sie kann Eingaben erhalten und ein Ergebnis zurückgeben.

Eine Methode definieren

```
int ggt(int a, int b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}
```

Methodenkopf

`int`: Typ des Rückgabewerts
`ggt`: Name der Methode
`int a, int b`: Parameter

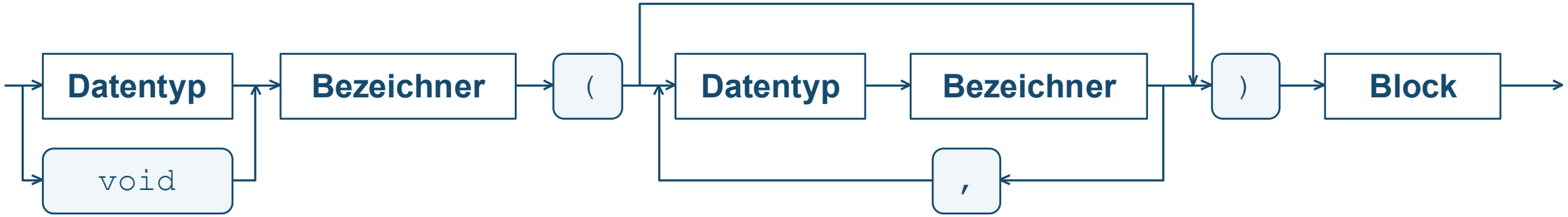
Methodenrumpf

Der Block enthält den Algorithmus.
`return a;` gibt das Ergebnis zurück.

Java 25: Methodendeklarationen

Syntax einer Methodendeklaration

<Methodendeklaration>




```
int addiere(int a, int b) { return a + b; }
```

Jeder Parameter braucht einen Datentyp und beim Aufruf einen passenden Wert.
`void`: kein Rückgabewert. Der Block enthält die Anweisungen in `{` und `}`.
Methodennamen beginnen üblicherweise klein, z. B. `addiere` oder `zeigeMenue`.

Gezeigter Ausschnitt ohne Arrays, Generics und weitere Zusätze · vollständige Syntax

Extract Method: vorher und nachher

Professionelle IDEs können markierten Code automatisch in eine eigene Methode auslagern.
In IntelliJ wählen wir Refactor → Extract → Method und vergeben den Namen `addiere`.

Vorher: die Berechnung markieren

```
void main() {
    int a = 4;
    int b = 6;
    int summe = a + b;
    IO.println(summe);
}
```

Nachher: die neue Methode nutzen

```
void main() {
    int a = 4;
    int b = 6;
    int summe = addiere(a, b);
    IO.println(summe);
}

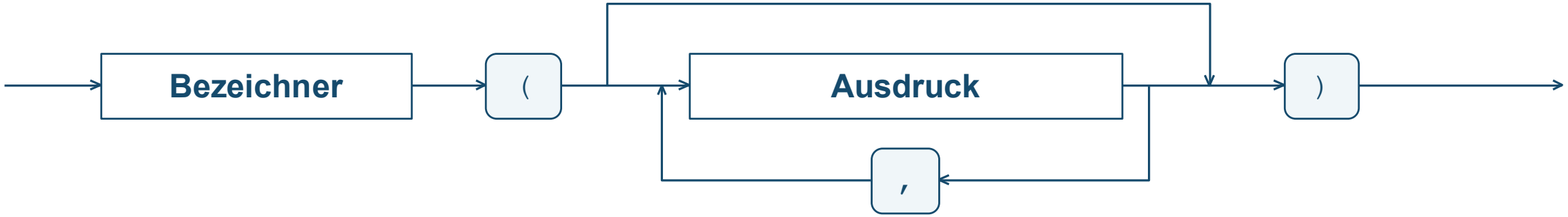
int addiere(int a, int b) {
    int summe = a + b;
    return summe;
}
```

Die IDE erzeugt Aufruf, Parameter und Rückgabe. Beide Programme geben `10` aus.

IntelliJ IDEA: Extract Method

Syntax eines Methodenaufrufs

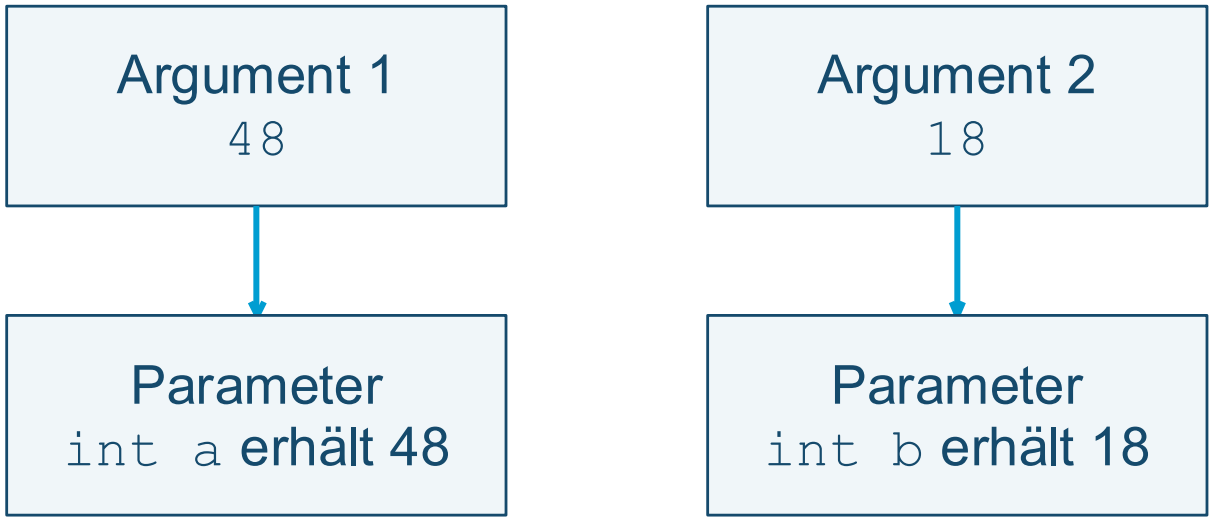
<Methodenaufruf> einer Methode in derselben Datei



`zeigeMenue();` steht als eigene Anweisung. Es wird kein Rückgabewert verwendet.
`int t = ggt(48, 18);` nutzt das Ergebnis. Das `;` beendet jeweils die ganze Anweisung.

Argumente und Parameter

```
int ergebnis = ggt(48, 18);
```



Argumente stehen beim Aufruf.
Parameter stehen in der Definition.

Die Reihenfolge ordnet sie zu.

Methodenkopf

```
int ggt(int a, int b)
```

Ein Parameter deklariert eine Variable der Methode. Beim Aufruf erhält sie eine Kopie des Wertes.
Über diese Variable kann die Methode den übergebenen Wert verwenden.

Die Parameternamen dürfen anders heißen

In `main` heißen die Variablen `a` und `b`. Die Methode verwendet die Parameter `x` und `y`.

Beim Aufruf

```
void main() {
    int a = 48;
    int b = 18;
    int ergebnis = ggt(a, b);
    IO.println(ergebnis); // 6
}
```

1. Argument: Wert von `a` als Kopie in `x`.
2. Argument: Wert von `b` als Kopie in `y`.

In der Methode

```
int ggt(int x, int y) {
    while (x != y) {
        if (x > y) {
            x = x - y;
        } else {
            y = y - x;
        }
    }
    return x;
}
```

Hießen die Parameter in `ggt` ebenfalls `a` und `b`, wären es trotzdem unabhängige Variablen.
In `main`: sichtbar ab ihrer Deklaration bis zur schließenden Blockklammer `}`.
In `ggt`: sichtbar nur innerhalb der Blockklammern `{ ... }` des Methodenrumpfs.

Wann braucht eine Methode einen Parameter?

Was braucht die Methode, um ihren Job erledigen zu können?

Auftrag	Benötigte Information	Passender Methodenkopf
Quadrat einer Zahl berechnen	Welche Zahl?	`int quadrat(int zahl)`
Text mehrfach ausgeben	Welcher Text? Wie oft?	`void wiederhole(String text, int anzahl)`
Ein festes Menü anzeigen	Keine wechselnde Eingabe	`void zeigeMenue()`

Welche Information hat sie bereits? Was fehlt noch und muss der Aufrufer liefern?
Zwischenwerte berechnet sie selbst. Dafür braucht sie keine Parameter.

Eingaben und Ergebnisse bewusst trennen

Einlesen beim Aufrufer

```
void main() {
    int a = Integer.parseInt(
        IO.readln("a: "));
    int b = Integer.parseInt(
        IO.readln("b: "));
    int ergebnis = ggt(a, b);
    IO.println(ergebnis);
}
```

Berechnen in der Methode

```
int ggt(int a, int b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}
```

Für jeden Parameter darf beim Aufruf ein beliebiger Java-Ausdruck stehen.
Sein Wert muss zum Parametertyp passen; eine erlaubte implizite Umwandlung ist möglich.
Beispiele: `ggt(a, b)`, `ggt(48, 18)` oder `ggt(a + 1, b)`.

Mit und ohne Parameter oder Rückgabewert

	Kein Rückgabewert: `void`	Mit Rückgabewert
Ohne Parameter	`void zeigeMenue()` zeigt ein festes Menü	`int liesZahl()` liest eine Zahl ein und liefert sie
Mit Parametern	`void zeigeZahl(int zahl)` gibt die Zahl aus	`int quadrat(int zahl)` berechnet und liefert ihr Quadrat

Keine Parameter bedeutet: Der Aufrufer übergibt keine Argumente.
Eine Methode kann trotzdem selbst Eingaben lesen oder andere Seiteneffekte haben.

Vier Kombinationen als Java-Code

Ohne Parameter, kein Rückgabewert

```
void zeigeMenue() {
    IO.println("1: Start");
    IO.println("0: Ende");
}
```

Ohne Parameter, mit Rückgabewert

```
int liesZahl() {
    String text = IO.readln("Zahl: ");
    return Integer.parseInt(text);
}
```

Mit Parameter, kein Rückgabewert

```
void zeigeZahl(int zahl) {
    IO.println(zahl);
}
```

Mit Parameter und Rückgabewert

```
int quadrat(int zahl) {
    return zahl * zahl;
}
```

Rückgabe mit `return`

<Rückgabeanweisung>



Methode mit Rückgabewert

```
int quadrat(int zahl) {
    return zahl * zahl;
}
```

Methode ohne Rückgabewert

```
void zeigeMenue() {
    IO.println("1: Start");
    return;
}
```

`return` beendet den aktuellen Methodenaufruf. Bei `void` entfällt der Ausdruck.

Java 25: return-Anweisung

Ein Ergebnis zurückgeben oder ausgeben

Rückgabe an den Aufrufer

```
int quadrat(int zahl) {
    return zahl * zahl;
}

// In main:
int wert = quadrat(5);
IO.println(wert + 1); // 26
```

Der Aufrufer kann mit 25 weiterrechnen.

Ausgabe auf dem Bildschirm

```
void zeigeQuadrat(int zahl) {
    IO.println(zahl * zahl);
}

// In main:
zeigeQuadrat(5); // Ausgabe: 25
```

Die Ausgabe ist ein Seiteneffekt.
Es kommt kein Wert zum Aufrufer zurück.

Eine Methode kann beides tun. Ein Bildschirmtext ersetzt aber keinen Rückgabewert.

Der Methodenaufruf als Ausdruck

```
int wert = quadrat(5);
int ergebnis = quadrat(5) + 1;
IO.println(quadrat(5));

quadrat(5);  // berechnet 25, verwirft aber den Rückgabewert
```

Nach dem Aufruf wird der Ausdruck mit dem Rückgabewert weiter ausgewertet.
Gedanklich ersetzt dieser Wert den Aufruf. Das geschieht zur Laufzeit.

```
quadrat(5) + 1   →   25 + 1   →   26
```

Der Aufruf hat den Ergebnistyp der Methode.

Ein `void`-Aufruf liefert keinen Wert.
Deshalb ist zum Beispiel `int x = zeigeQuadrat(5);` nicht erlaubt.

Java 25: Ausdrücke und verworfene Rückgabewerte

Eine `void`-Methode vorzeitig verlassen

Aufrufer: `main`

```
void main() {
    begruesse("");
    IO.println("Weiter in main.");
}
```

Bildschirmausgabe

```
Kein Name angegeben.
Weiter in main.
```

Aufgerufene Methode

```
void begruesse(String name) {
    if (name.equals("")) {
        IO.println(
            "Kein Name angegeben.");
        return;
    }
    IO.println("Hallo " + name + "!");
}
```

`return;` beendet nur den Aufruf von `begruesse`.
Danach läuft `main` mit `IO.println("Weiter in main.");` weiter.

Bei einer void-Methode ist return; erlaubt. Am Ende des Rumpfs kann es entfallen.

Ein Ergebnis auf jedem normalen Rückweg

Hier fehlt eine Rückgabe

```
int maximum(int a, int b) {
    if (a > b) {
        return a;
    }
    // und bei a <= b?
}
```

Compilerfehler: Das Ende des Rumpfs kann ohne Rückgabewert erreicht werden.

Vollständig

```
int maximum(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
```

Nach `return a;` geht es direkt zum Aufrufer zurück.

Java 25: Methodenrumpf und Rückgabewert

`return` in einer Suchschleife

Gesucht: ein Teiler zwischen 2 und `zahl - 1`. Beispiel: 9 hat den Teiler 3.

Zu früh beendet

```
boolean hatTeiler(int zahl) {
    for (int t = 2; t < zahl; t++) {
        if (zahl % t == 0) {
            return true;
        } else {
            return false;
        }
    }
    return false;
}
```

`hatTeiler(9)` liefert fälschlich `false`:
Nach dem Test mit 2 endet die Methode.
Der Teiler 3 kommt nie an die Reihe.

Alle Kandidaten prüfen

```
boolean hatTeiler(int zahl) {
    for (int t = 2; t < zahl; t++) {
        if (zahl % t == 0) {
            return true;
        }
    }
    return false;
}
```

`hatTeiler(9)` liefert `true` beim Teiler 3.
`false` folgt erst nach der Schleife,
wenn kein Teiler gepasst hat.

Ein vollständiges Compact File

```
void main() {
    IO.println(ggt(48, 18));
    IO.println(ggt(35, 14));
}

int ggt(int a, int b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}
```

Datei: `Ggt.java`

Die Methodendefinitionen stehen nebeneinander.

Eine Definition steht nicht innerhalb von `main`.

Java 25 benötigt hier keine ausgeschriebene Klasse und kein `static`.

Java 25: Compact Source Files

Definition und Ausführung

```
void main() {
    IO.println("Start");
    zeigeHinweis();
    IO.println("Ende");
}

void zeigeHinweis() {
    IO.println("Hier ist der Hinweis.");
}
```

Ausgabe

```
Start
Hier ist der Hinweis.
Ende
```

Die Definition legt fest, was die Methode tut. Erst der Aufruf führt ihren Code aus.

Die Reihenfolge der Methodendefinitionen in der Datei legt die Aufrufreihenfolge nicht fest.

Parameter und lokale Variablen

Im Aufrufer

```
void main() {
    int zahl = 5;
    int wert = quadrat(zahl);
    IO.println(wert);
}
```

Der Aufrufer kennt seine Variablen `zahl` und `wert`.

In der aufgerufenen Methode

```
int quadrat(int zahl) {
    int ergebnis = zahl * zahl;
    return ergebnis;
}
```

Die Methode kennt ihren Parameter `zahl` und ihre lokale Variable `ergebnis`.

Gleiche Namen erzeugen keine gemeinsame Variable.
Lokale Variablen einer Methode sind in anderen Methoden nicht sichtbar.

Variablen außerhalb der Methoden

```
int zaehler = 0;

void erhoehe() {
    zaehler++;
}

void main() {
    erhoehe();
    erhoehe();
    IO.println(zaehler); // 2
}
```

Vorteil

Mehrere Methoden nutzen denselben Zähler und behalten seinen Wert zwischen den Aufrufen.

Nachteile

Die Änderung von `zaehler` ist ein Seiteneffekt. Er bleibt im Aufruf verborgen. Frühere Aufrufe beeinflussen den Wert. Das erschwert Fehlersuche und Tests.

Empfehlung: Zwischenwerte lokal halten. Daten über Parameter und Rückgabewerte austauschen. Übergeordnete Variablen nur für bewusst gemeinsamen Zustand verwenden.

Java 25: Methoden und Variablen in Compact Source Files

Startwerte: übergeordnete und lokale Variablen

```
int anzahl; // automatisch 0

void main() {
    int versuche; // noch kein Wert

    IO.println(anzahl); // 0
    // IO.println(versuche); // Fehler

    versuche = 3;
    IO.println(versuche); // 3
}
```

Standardwerte

Datentyp	Standardwert
`byte, short, int, long`	`0`
`float`	`0.0f`
`double`	`0.0`
`boolean`	`false`
`char`	`'\u0000'`
`String`	`null`

Übergeordnete Variablen erhalten automatisch einen Standardwert.
Lokalen Variablen müssen wir vor dem ersten Lesen selbst einen Wert zuweisen.

Java-Sprachspezifikation: Anfangswerte von Variablen

Mathematische Funktionen und Seiteneffekte

Beispiel	Ergebnis	Weitere Wirkung
`quadrat(5)`	`25`	Keine
`IO.println(25)`	Kein Wert (`void`)	Text erscheint auf dem Bildschirm
`IO.readln("Zahl: ")`	Eingelesener `String`	Eingabe wird gelesen, Aufforderung ausgegeben
`erhoehe()`	Kein Wert (`void`)	Ändert die gemeinsame Variable

Eine reine Berechnung liefert bei gleichen Eingaben dasselbe Ergebnis
und verändert außerhalb des Aufrufs nichts. Das erleichtert Verstehen und Prüfen.

Methoden mit einem Doc-Comment dokumentieren

```
/**
 * Liefert den größeren von zwei ganzzahligen Werten.
 *
 * @param a erster Wert
 * @param b zweiter Wert
 * @return größerer Wert; bei Gleichheit der gemeinsame Wert
 */
int maximum(int a, int b) {
    return a > b ? a : b;
}
```

Der Kommentar steht unmittelbar vor der Methode.
Er beschreibt Auftrag, Voraussetzungen und Ergebnis für die Personen, die sie aufrufen.

Java 25: Javadoc-Kommentare und Tags

Wichtige Javadoc-Tags

Tag	Bedeutung
`@param name Beschreibung`	Erklärt einen Parameter; für jeden eine Zeile.
`@return Beschreibung`	Beschreibt das Ergebnis; entfällt bei `void`.
`{@code Ausdruck}`	Zeigt Code innerhalb des Beschreibungstextes.
`{@link Math#sqrt(double)}`	Verlinkt ein dokumentiertes Programmelement.
`@see Math` / `@since 1.0`	Weiterer Verweis / seit welcher Version vorhanden.
`@throws …`	Mögliche Ausnahmen; Details folgen später.

Das sind Tags, keine Java-Annotationen. `@returns` und `@type` sind keine Standard-Tags. Die Datentypen stehen bereits im Methodenkopf.

Vollständige Liste der Javadoc-Tags

Dokumentation und verständliche Namen

Die Dokumentation beantwortet die Fragen des Aufrufers.

Was berechnet oder verändert die Methode?
Welche Eingaben sind erlaubt? Welche Einheit haben Zahlen?
Was bedeutet das Ergebnis? Gibt es besondere Fälle?

Namen machen den Auftrag sichtbar.

```
istSchaltjahr(jahr)
berechneZuschlag(grundpreis)
zeigeMenue()
```

Methodennamen beginnen üblicherweise klein und folgen lowerCamelCase.
Ein Verb beschreibt oft die Tätigkeit.

Rename: Methodennamen überall anpassen

IntelliJ: `rechne` auswählen, Refactor → Rename (Shift+F6) und `addiere` eingeben.

Vorher: `rechne`

```
void main() {
    IO.println(rechne(4, 6));
    IO.println(rechne(2, 3));
}

int rechne(int a, int b) {
    return a + b;
}
```

Nachher: `addiere`

```
void main() {
    IO.println(addiere(4, 6));
    IO.println(addiere(2, 3));
}

int addiere(int a, int b) {
    return a + b;
}
```

Die IDE ändert den Namen in der Definition und in beiden Aufrufen.

Auch Variablen so umbenennen: Die IDE passt ihre zugehörigen Verwendungen an.

IntelliJ IDEA: Rename Refactoring

Methoden überladen

Mehrere Methoden können denselben Namen und unterschiedliche Parameterlisten haben.

Zwei Ganzzahlen

```
int maximum(int a,
            int b) {
    return a > b ? a : b;
}
```

Drei Ganzzahlen

```
int maximum(int a,
            int b,
            int c) {
    int m = a > b ? a : b;
    return m > c ? m : c;
}
```

Zwei Kommazahlen

```
double maximum(double a,
               double b) {
    return a > b ? a : b;
}
```

```
maximum(3, 7);        // zwei int-Parameter: 7
maximum(3, 7, 5);     // drei int-Parameter: 7
maximum(3.5, 7.2);    // zwei double-Parameter: 7.2
```

Java 25: Überladen von Methoden

Welche Parameterliste unterscheidet Methoden?

Unterschied	Überladen möglich?	Beispiel
Anzahl der Parameter	Ja	`f(int a)` / `f(int a, int b)`
Typen oder Reihenfolge	Ja	`f(int a, double b)` / `f(double a, int b)`
Nur Parameternamen	Nein	`f(int a)` / `f(int zahl)`
Nur Rückgabetyp	Nein	`int f(int a)` / `double f(int a)`

In unseren Beispielen besteht die Signatur aus Methodenname
und den Parametertypen in ihrer Reihenfolge. Der Rückgabetyp gehört nicht dazu.

Java 25: Methodensignatur

Auswahl einer überladenen Methode

```
maximum(3, 7);          // int, int: passende int-Variante
maximum(3, 7.2);        // int wird zu double erweitert
maximum(3.0, 7.0);      // double-Variante, auch ohne Nachkommarest
```

Der Compiler verwendet die Typen der Argumente.

Bei mehreren passenden Methoden wählt er die spezifischste.
Gibt es keine eindeutige Wahl, meldet er einen Compilerfehler.
Eine notwendige Verengung von `double` zu `int` geschieht nicht automatisch.

Java 25: Auswahl einer Methodendeklaration beim Aufruf

Gemeinsam: die passende Methodenschnittstelle

Ein Programm soll für mehrere Jahreszahlen feststellen, ob es Schaltjahre sind.
Bis 1581 gilt in unserem Modell die julianische Regel, ab 1582 die gregorianische.

Fragen in die Runde

1. Welcher Methodenname passt zum Auftrag?
2. Welche Parameter benötigt die Methode?
3. Welchen Ergebnistyp wählen Sie?
4. Wo gehören Einlesen und Bildschirmausgabe hin?

Wie könnte der Methodenkopf aussehen?
Was sollte der Doc-Comment erklären?

Ein möglicher Entwurf für das Schaltjahr

```
/**
 * Prüft ein positives Jahr: bis 1581 julianisch, ab 1582 gregorianisch.
 * @param jahr zu prüfende Jahreszahl, größer als 0
 * @return true, wenn das Jahr nach diesem Modell ein Schaltjahr ist
 */
boolean istSchaltjahr(int jahr) {
    if (jahr < 1582) {
        return jahr % 4 == 0;
    }
    return jahr % 400 == 0
        || (jahr % 4 == 0 && jahr % 100 != 0);
}
```

Der Aufrufer kann das Ergebnis speichern, in einer Bedingung nutzen oder ausgeben.

Methoden gezielt einsetzen

Vor dem Schreiben einer Methode

Den Auftrag in einem Satz beschreiben.
Eingaben als Parameter und Ergebnisse als Rückgabewert festlegen.
Seiteneffekte bewusst wählen und dokumentieren.

Beim Aufrufen einer Methode

Passende Argumente in der richtigen Reihenfolge übergeben.
Den Rückgabewert verwenden, wenn das Ergebnis gebraucht wird.
Die Methodensignatur und ihre Dokumentation beachten.

Was bei Methodenaufrufen passiert

Wertübergabe verstehen

Was erhält ein Parameter? Was verändert sich beim Aufrufer?

Aufrufe im Laufzeitstack verfolgen

Wo bleiben lokale Werte und noch nicht abgeschlossene Berechnungen?

Rekursion und Endrekursion

Wie kann eine Methode sich selbst aufrufen und wieder fertig werden?

Wertübergabe: Was wird hier ausgegeben?

Aufrufer

```
void main() {
    int zahl = 5;
    verdopple(zahl);
    IO.println(zahl);
}
```

Aufgerufene Methode

```
void verdopple(int wert) {
    wert = wert * 2;
    IO.println(wert);
}
```

Vorhersage: Welche beiden Zahlen erscheinen?
Verändert die Zuweisung an `wert` auch die Variable `zahl`?

Java übergibt eine Kopie des Werts

Methodenaufruf in `main`

```
int zahl = 5;
verdopple(zahl);
```

Methodenkopf

```
void verdopple(int wert)
```

Argument `zahl`
Wert: `5`

↓

Parameter: `int wert`
`5` (Kopie)

Beim Aufruf erhält `wert` eine Kopie der `5`.
`zahl` und `wert` sind unabhängige Variablen.

```
wert = wert * 2;
```

In `main` unverändert
`zahl = 5`

In `verdopple` geändert
`wert = 10`

Ausgabe wie auf der vorigen Folie: erst `10` in `verdopple`, danach `5` in `main`.

Call-by-Value: neue Parametervariablen erhalten die Werte der Argumente.

Ein neues Ergebnis beim Aufrufer übernehmen

```
int verdopple(int wert) {
    return wert * 2;
}

void main() {
    int zahl = 5;
    zahl = verdopple(zahl);
    IO.println(zahl); // 10
}
```

Die Methode liefert 10 zurück.

Erst die Zuweisung im Aufrufer verändert dessen Variable `zahl`.

Ein Parameter transportiert hier eine Eingabe. `return` transportiert das Ergebnis.

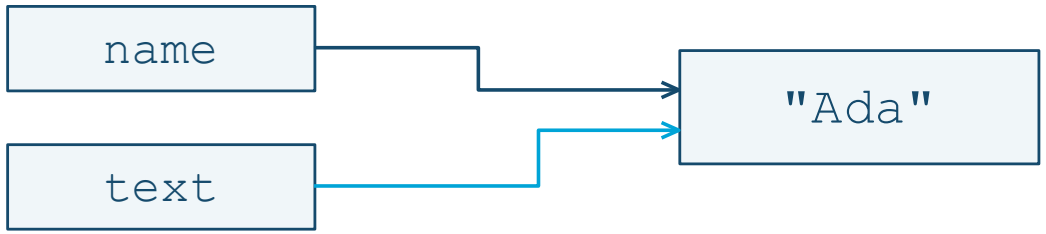
Auch bei `String` wird ein Wert kopiert

Eine `String`-Variable speichert eine Referenz auf den Text im Speicher.
Beim Aufruf erhält der Parameter eine Kopie dieser Referenz.

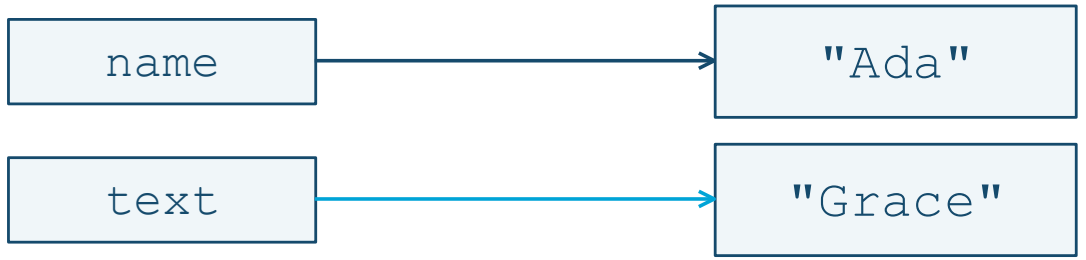
```
void main() {
    String name = "Ada";
    aendere(name);
    IO.println(name); // Ada
}
```

```
void aendere(String text) {
    text = "Grace";
}
```

Am Anfang von `aendere`



Nach der Zuweisung, vor der Rückkehr



Nach der Rückkehr entfällt `text`. `name` zeigt weiter auf `"Ada"`.

Java 25: Parameterübergabe

Call-by-Value und Call-by-Reference

Java: Call-by-Value

```
void main() {
    int zahl = 5;
    verdopple(zahl);
    IO.println(zahl); // 5
}

void verdopple(int wert) {
    wert = wert * 2;
    IO.println(wert); // 10
}
```

Ausgabe: `10`, dann `5`

`wert` erhält eine Kopie von `zahl`.
Die Zuweisung ändert nur `wert`.
`zahl` bleibt `5`.

Java 25: Parameterübergabe

C#: Call-by-Reference mit `ref`

```
using System;

int zahl = 5;
Verdopple(ref zahl);
Console.WriteLine(zahl); // 10

void Verdopple(ref int wert) {
    wert = wert * 2;
    Console.WriteLine(wert); // 10
}
```

Ausgabe: `10`, dann `10`

`ref` macht `wert` zu einem zweiten Namen für `zahl`. Ohne `ref` nutzt auch C# Call-by-Value.

C#: Parameterübergabe mit ref

Die Wertübergabe im Debugger beobachten

In IntelliJ IDEA mit dem Java-Beispiel der vorherigen Folie

1. Breakpoint auf `verdopple(zahl);` setzen und „Debug“ starten.
2. Mit „Step Into“ in `verdopple` wechseln: `wert` hat zunächst den Wert `5`.
3. Mit „Step Over“ die Zuweisung ausführen: `wert` ist danach `10`.
4. Im Bereich „Frames“ `main` auswählen: Dort ist `zahl` weiterhin `5`.
5. Zurück zu `verdopple` wechseln. „Step Out“ führt zurück zu `main`.
6. In `main` die letzte Ausgabe ausführen: `zahl` ist weiterhin `5`.

Wir beobachten zwei getrennte Variablen: `wert` in `verdopple` und `zahl` in `main`.

IntelliJ IDEA: schrittweise Ausführung im Debugger

Methodenaufrufe im Laufzeitstack

1. `main` ist aktiv. Der Aufruf von `plusEins` steht als Nächstes an.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int wert = plusEins(3);
3      IO.println(wert);
4  }
5
6  int plusEins(int x) {
7      int y = doppelt(x);
8      return y + 1;
9  }
10
11 int doppelt(int x) {
12     return 2 * x;
13 }
```

Laufzeitstack

Oben liegt der aktive Aufruf.

`main()`
`wert: noch kein Ergebnis`
vor dem Aufruf von `plusEins`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Methodenaufrufe im Laufzeitstack

2. `plusEins` wird aufgerufen. Ein neuer Eintrag mit `x = 3` kommt hinzu.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int wert = plusEins(3);
3      IO.println(wert);
4  }
5
6  int plusEins(int x) {
7      int y = doppelt(x);
8      return y + 1;
9  }
10
11 int doppelt(int x) {
12     return 2 * x;
13 }
```

Laufzeitstack

Oben liegt der aktive Aufruf.

`plusEins(3)`
`x = 3; y noch ohne Wert`
vor dem Aufruf von `doppelt`

`main()`
`wartet auf plusEins`
danach: Zuweisung in Zeile 2

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Methodenaufrufe im Laufzeitstack

3. `doppelt` wird aufgerufen. Es erhält einen weiteren Eintrag mit einer eigenen Variablen `x`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int wert = plusEins(3);
3      IO.println(wert);
4  }
5
6  int plusEins(int x) {
7      int y = doppelt(x);
8      return y + 1;
9  }
10
11 int doppelt(int x) {
12     return 2 * x;
13 }
```

Laufzeitstack

Oben liegt der aktive Aufruf.

`doppelt(3)`
`x = 3`
als Nächstes: `return 2 * x;`

`plusEins(3)`
`x = 3; y noch ohne Wert`
wartet auf `doppelt`

`main()`
`wartet auf plusEins`
danach: Zuweisung in Zeile 2

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Methodenaufrufe im Laufzeitstack

4. `doppelt` liefert 6. Sein Eintrag verschwindet und `plusEins` übernimmt 6 in `y`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int wert = plusEins(3);
3      IO.println(wert);
4  }
5
6  int plusEins(int x) {
7      int y = doppelt(x);
8      return y + 1;
9  }
10
11 int doppelt(int x) {
12     return 2 * x;
13 }
```

Laufzeitstack

Oben liegt der aktive Aufruf.

`plusEins(3)`
`x = 3; y = 6`
als Nächstes: `return y + 1;`

`main()`
`wartet auf plusEins`
danach: Zuweisung in Zeile 2

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Methodenaufrufe im Laufzeitstack

5. `plusEins` liefert 7. Sein Eintrag verschwindet und `main` übernimmt 7 in `wert`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int wert = plusEins(3);
3      IO.println(wert);
4  }
5
6  int plusEins(int x) {
7      int y = doppelt(x);
8      return y + 1;
9  }
10
11 int doppelt(int x) {
12     return 2 * x;
13 }
```

Laufzeitstack

Oben liegt der aktive Aufruf.

`main()`
`wert = 7`
als Nächstes: Bildschirmausgabe

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Methodenaufrufe im Laufzeitstack

6. Die Ausgabe ist erfolgt. Auch der Aufruf von `main` ist beendet.

Code: `main` ist abgeschlossen

```
1  void main() {
2      int wert = plusEins(3);
3      IO.println(wert);
4  }
5
6  int plusEins(int x) {
7      int y = doppelt(x);
8      return y + 1;
9  }
10
11 int doppelt(int x) {
12     return 2 * x;
13 }
```

Laufzeitstack

Kein offener Aufruf unserer drei Methoden mehr.

Ausgabe: 7

Grau markiert: das Ende von `main`.

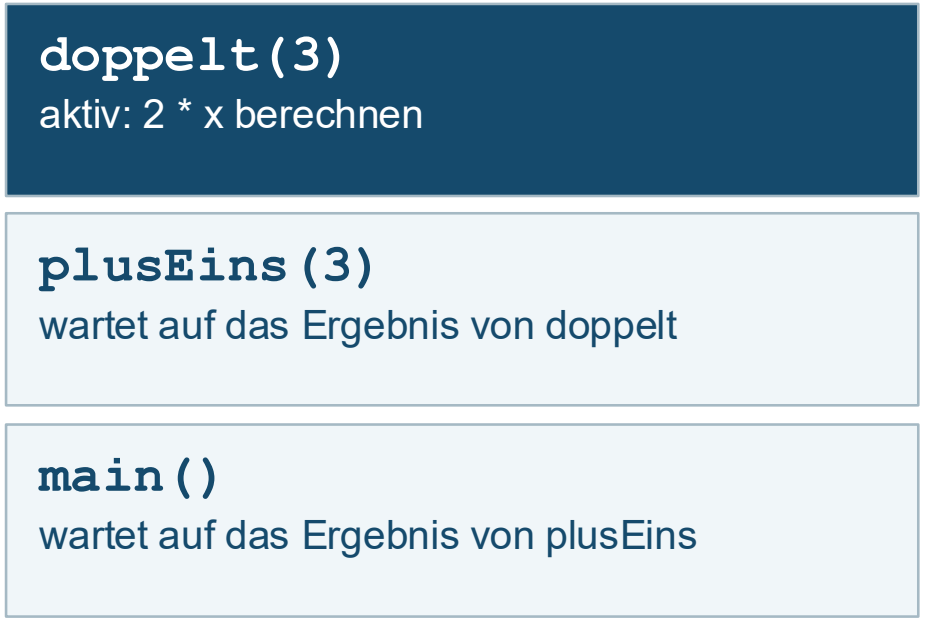
Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Der Laufzeitstack

Ein Stack ist ein Stapel: Der zuletzt hinzugefügte Eintrag kommt zuerst wieder herunter.

Ein Eintrag pro Methodenaufruf

Aufruf: Ein neuer Stackframe kommt oben hinzu.
Rückkehr: Der oberste Frame verschwindet.
Der Aufrufer setzt seine Arbeit fort.



Vereinfachtes Modell des JVM-Stacks für unseren Programmablauf

Was zu einem Aufruf gehört

Stackframe: `plusEins(3)`

Parameter: `x = 3`
Lokale Variablen: zum Beispiel `y`
Zwischenergebnisse von Berechnungen
Informationen zur aufgerufenen Methode

Rücksprungadresse

In `main` nach dem Aufruf `plusEins(3)`:
Rückgabewert in `wert` speichern.

Wo geht es danach weiter?

Der Aufruf in `main`:

```
int wert = plusEins(3);
```

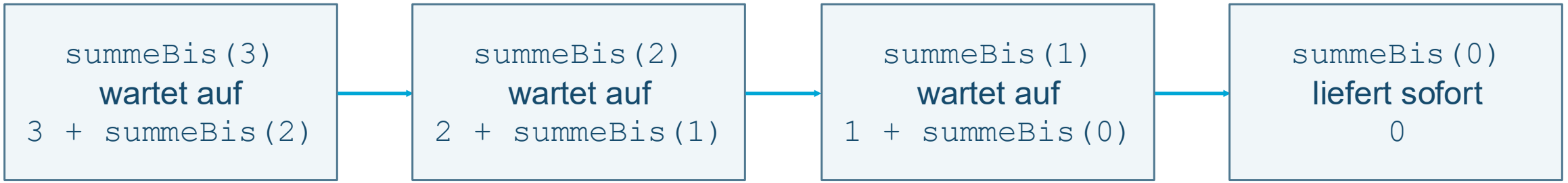
Die Rücksprungadresse zeigt auf die Fortsetzungsstelle im Aufrufer.
Nach der Zuweisung folgt die nächste Anweisung in `main`.

Die konkrete Speicherung hängt von der JVM und ihrer Umsetzung ab.
Die Grafik zeigt ein Modell, keinen fest vorgeschriebenen Speicheraufbau.

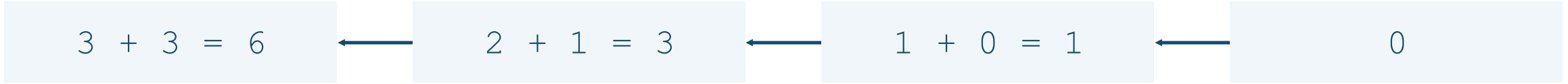
JVM: lokale Variablen, Operandenstack und normale Rückkehr

Rekursion: ein kleineres Problem derselben Art

Wir möchten die Zahlen von `1` bis `n` addieren.
Für `n > 0`: `summeBis` ruft sich mit `n - 1` selbst auf und wartet auf das Ergebnis.



Jeder wartende Aufruf addiert seine Zahl zum zurückgegebenen Ergebnis:



Abbruchfall: `summeBis(0)` liefert `0`, ohne sich erneut aufzurufen.

Eine rekursive Methode

```
int summeBis(int n) {
    if (n == 0) {
        return 0;
    }
    return n + summeBis(n - 1);
}
```

```
IO.println(summeBis(3)); // 6
```

Voraussetzung: n ≥ 0

Abbruchfall:
`n == 0`

Rekursiver Schritt:
Aufruf mit `n - 1`

Das Ergebnis wird noch zu `n` addiert.

Rekursion im Laufzeitstack

1. `main` ist aktiv. Als Nächstes wird `summeBis(3)` aufgerufen.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`main()`
`ergebnis`: noch kein Wert

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Rekursion im Laufzeitstack

2. `summeBis(3)` ist aktiv. `n == 0` ist falsch, daher folgt der Aufruf mit `n = 2`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(3)`
`n = 3`; ruft `summeBis(2)` auf

`main()`
wartet auf `summeBis(3)`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Rekursion im Laufzeitstack

3. Ein weiterer Aufruf ist aktiv: `summeBis(2)`. Nach der Prüfung folgt der Aufruf mit `n = 1`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(2)`
`n = 2`; ruft `summeBis(1)` auf

`summeBis(3)`
`n = 3`; wartet auf `3 + …`

`main()`
wartet auf `summeBis(3)`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Rekursion im Laufzeitstack

4. Nun ist `summeBis(1)` aktiv. Nach der Prüfung folgt der Aufruf mit `n = 0`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(1)`
`n = 1`; ruft `summeBis(0)` auf

`summeBis(2)`
`n = 2`; wartet auf `2 + …`

`summeBis(3)`
`n = 3`; wartet auf `3 + …`

`main()`
wartet auf `summeBis(3)`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Rekursion im Laufzeitstack

5. `summeBis(0)` erreicht den Abbruchfall. `n == 0` ist wahr, daher wird jetzt 0 zurückgegeben.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(0)`
`n = 0`; gibt 0 zurück

`summeBis(1)`
`n = 1`; wartet auf `1 + …`

`summeBis(2)`
`n = 2`; wartet auf `2 + …`

`summeBis(3)`
`n = 3`; wartet auf `3 + …`

`main()`
wartet auf `summeBis(3)`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Rekursion im Laufzeitstack

6. `summeBis(0)` ist beendet. `summeBis(1)` erhält 0 und setzt die Addition fort: `1 + 0 = 1`.

Code: blaue Zeile fortsetzen

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(1)`
`n = 1`; gibt `1 + 0 = 1` zurück

`summeBis(2)`
`n = 2`; wartet auf `2 + …`

`summeBis(3)`
`n = 3`; wartet auf `3 + …`

`main()`
wartet auf `summeBis(3)`

Der rekursive Aufruf ist bereits beendet. Jetzt folgen die Addition und die Rückgabe.

Rekursion im Laufzeitstack

7. `summeBis(1)` ist beendet. `summeBis(2)` erhält 1 und setzt die Addition fort: `2 + 1 = 3`.

Code: blaue Zeile fortsetzen

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(2)`
`n = 2`; gibt `2 + 1 = 3` zurück

`summeBis(3)`
`n = 3`; wartet auf `3 + …`

`main()`
wartet auf `summeBis(3)`

Der rekursive Aufruf ist bereits beendet. Jetzt folgen die Addition und die Rückgabe.

Rekursion im Laufzeitstack

8. `summeBis(2)` ist beendet. `summeBis(3)` erhält 3 und setzt die Addition fort: `3 + 3 = 6`.

Code: blaue Zeile fortsetzen

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`summeBis(3)`
`n = 3`; gibt `3 + 3 = 6` zurück

`main()`
wartet auf `summeBis(3)`

Der rekursive Aufruf ist bereits beendet. Jetzt folgen die Addition und die Rückgabe.

Rekursion im Laufzeitstack

9. `summeBis(3)` ist beendet. `main` übernimmt das Ergebnis 6 und führt als Nächstes die Ausgabe aus.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

`main()`
`ergebnis` = 6; Ausgabe folgt

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Rekursion im Laufzeitstack

10. Die Ausgabe ist erfolgt. Auch `main` ist beendet, der Stack unserer Aufrufe ist leer.

Code: `main` ist abgeschlossen

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      if (n == 0) {
8          return 0;
9      }
10     return n + summeBis(n - 1);
11 }
```

Laufzeitstack (oben aktiv)

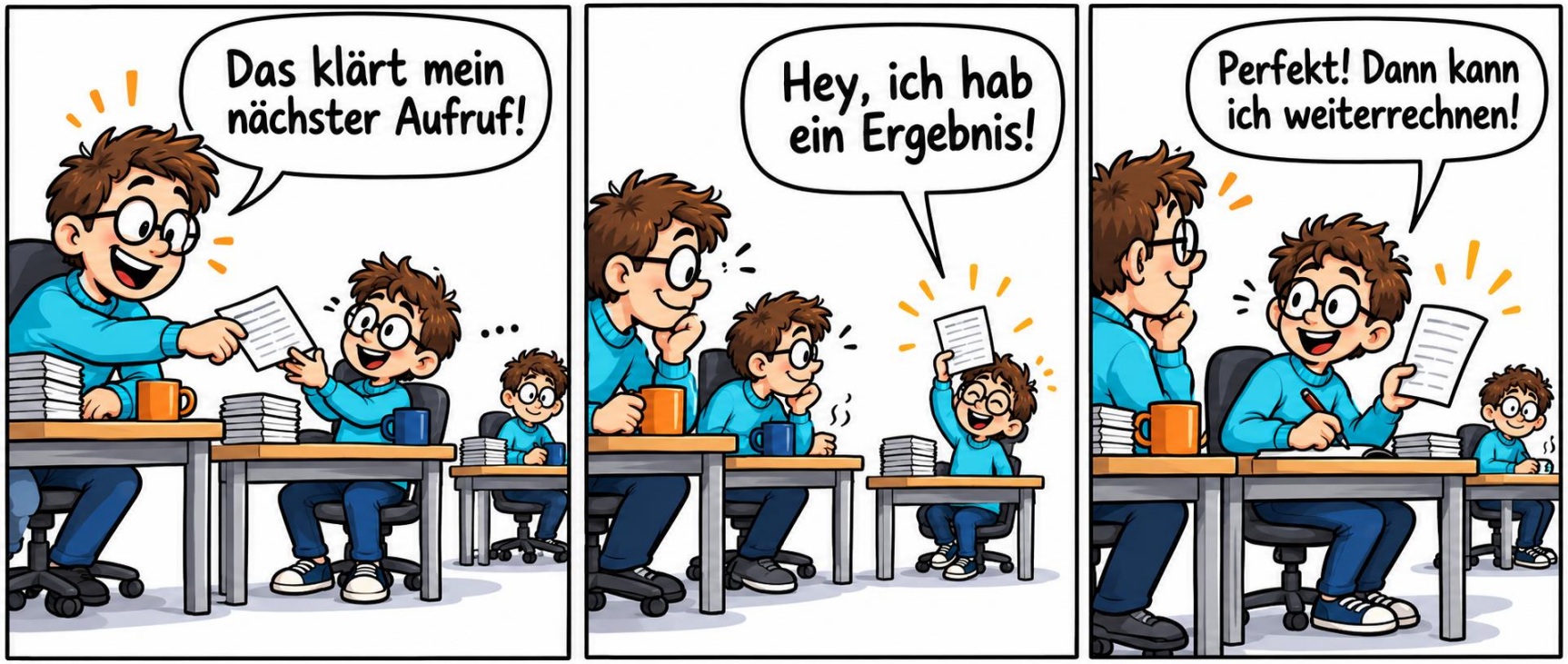
Alle Aufrufe sind beendet.

Ausgabe: 6

Grau markiert: das Ende von `main`.

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

„Hey, ich hab ein Ergebnis!“



Die Figuren stehen für einzelne Aufrufe. In unserem Programm arbeitet jeweils nur ein Aufruf, während seine Aufrufer auf das Ergebnis warten.

Wann endet eine Rekursion?

Ein erreichbarer Abbruchfall

Für diesen Fall liefert die Methode ein Ergebnis ohne weiteren Selbstaufruf.

Ein Schritt in Richtung Abbruchfall

Bei `summeBis(n)` wird n kleiner. Für n ≥ 0 erreicht die Folge schließlich 0.

Was bei falschen Eingaben passieren kann

Mit negativem n entfernt sich `n - 1` zunächst weiter von 0.
Zu viele offene Aufrufe können einen `StackOverflowError` auslösen.

JVM: begrenzter Stack und StackOverflowError

Der ggT als rekursive Methode

```
int ggt(int a, int b) {
    if (b == 0) {
        return a;
    }
    return ggt(b, a % b);
}
```

Voraussetzung

a > 0 und b ≥ 0

Der Rest `a % b` ist bei b > 0 kleiner als b.
Die zweite Zahl erreicht schließlich 0.

```
ggt(48, 18) → ggt(18, 12) → ggt(12, 6) → ggt(6, 0)
```

Der letzte Aufruf liefert 6. Alle vorherigen Aufrufe reichen diesen Wert zurück.

Endrekursion

Noch Arbeit nach dem Aufruf

```
return n + summeBis(n - 1);
```

Der Aufrufer muss anschließend noch n zum Ergebnis addieren.

Ergebnis direkt weiterreichen

```
return ggt(b, a % b);
```

Nach der Rückkehr wird das Ergebnis unverändert zurückgegeben.

Bei Endrekursion ist der rekursive Aufruf die letzte Aktion.
Sein Ergebnis wird anschließend nur noch unverändert zurückgegeben.

Die Summe endrekursiv formulieren

```
int summeBis(int n) {
    return summeBis(n, 0);
}

int summeBis(int n, int summe) {
    if (n == 0) {
        return summe;
    }
    return summeBis(n - 1, summe + n);
}
```

Der zusätzliche Parameter sammelt das bisherige Ergebnis.

Die Überladung mit einem Parameter setzt den Startwert 0.

Voraussetzung: $n \geq 0$.

Die Addition erfolgt vor dem nächsten Aufruf. Bei der Rückkehr bleibt nur Weiterreichen.

Das Zwischenergebnis wird mitgegeben

Aufruf	Bisherige Summe	Nächster Schritt
`summeBis(3, 0)`	0	3 addieren
`summeBis(2, 3)`	3	2 addieren
`summeBis(1, 5)`	5	1 addieren
`summeBis(0, 6)`	6	6 zurückgeben

Auf dem Rückweg reichen die Aufrufe die 6 unverändert weiter.
Auch dabei bleiben in Java normalerweise mehrere Stackframes offen.

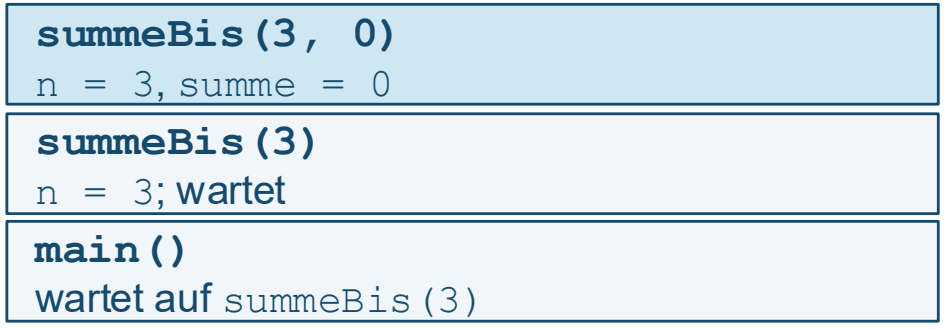
Endrekursion im Laufzeitstack

1. Die Hilfsmethode startet mit `summe = 0`. Vor dem nächsten Aufruf wird `0 + 3 = 3` berechnet.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      return summeBis(n, 0);
8  }
9
10 int summeBis(int n, int summe) {
11     if (n == 0) {
12         return summe;
13     }
14     return summeBis(n - 1, summe + n);
15 }
```

Laufzeitstack (oben aktiv)



Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Endrekursion im Laufzeitstack

2. Beim Aufbau wächst `summe` von 0 über 3 auf 5. Nun folgt die letzte Addition: `5 + 1 = 6`.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      return summeBis(n, 0);
8  }
9
10 int summeBis(int n, int summe) {
11     if (n == 0) {
12         return summe;
13     }
14     return summeBis(n - 1, summe + n);
15 }
```

Laufzeitstack (oben aktiv)

`summeBis(1, 5)`
`n = 1, summe = 5`

`summeBis(2, 3)`
`n = 2, summe = 3`; wartet

`summeBis(3, 0)`
`n = 3, summe = 0`; wartet

`summeBis(3)`
`n = 3`; wartet

`main()`
wartet auf `summeBis(3)`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Endrekursion im Laufzeitstack

3. Im Abbruchfall ist die Summe schon fertig: `summeBis(0, 6)` gibt 6 zurück.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      return summeBis(n, 0);
8  }
9
10 int summeBis(int n, int summe) {
11     if (n == 0) {
12         return summe;
13     }
14     return summeBis(n - 1, summe + n);
15 }
```

Laufzeitstack (oben aktiv)

`summeBis(0, 6)`
`n = 0, summe = 6`

`summeBis(1, 5)`
`n = 1, summe = 5`; wartet

`summeBis(2, 3)`
`n = 2, summe = 3`; wartet

`summeBis(3, 0)`
`n = 3, summe = 0`; wartet

`summeBis(3)`
`n = 3`; wartet

`main()`
wartet auf `summeBis(3)`

Vereinfachtes Modell: Aufrufe der Java-Bibliothek sind ausgeblendet.

Endrekursion im Laufzeitstack

4. Beim Abbau reicht `summeBis(1, 5)` die erhaltene 6 unverändert zurück.

Code: Rückgabewert weiterreichen

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      return summeBis(n, 0);
8  }
9
10 int summeBis(int n, int summe) {
11     if (n == 0) {
12         return summe;
13     }
14     return summeBis(n - 1, summe + n);
15 }
```

Laufzeitstack (oben aktiv)

`summeBis(1, 5)`
`n = 1`, `summe = 5`; Rückgabe: 6

`summeBis(2, 3)`
`n = 2`, `summe = 3`; wartet

`summeBis(3, 0)`
`n = 3`, `summe = 0`; wartet

`summeBis(3)`
`n = 3`; wartet

`main()`
wartet auf `summeBis(3)`

Die übrigen Aufrufe reichen dieselbe 6 zurück. Nach dem rekursiven Aufruf wird nicht mehr addiert.

Endrekursion im Laufzeitstack

5. Alle weiteren Aufrufe haben 6 zurückgereicht. `main` übernimmt das Ergebnis und gibt es aus.

Code: nächste Zeile blau markiert

```
1  void main() {
2      int ergebnis = summeBis(3);
3      IO.println(ergebnis);
4  }
5
6  int summeBis(int n) {
7      return summeBis(n, 0);
8  }
9
10 int summeBis(int n, int summe) {
11     if (n == 0) {
12         return summe;
13     }
14     return summeBis(n - 1, summe + n);
15 }
```

Laufzeitstack (oben aktiv)

`main()`
`ergebnis = 6;` Ausgabe folgt

Die nächste Zeile ist die Ausgabe. Danach endet auch `main`.

Rekursion und Endrekursion im Vergleich

Nicht endrekursiv

```
return n + summeBis(n - 1);
```

Beim Rückweg wird gerechnet:

```
1 + 0 = 1
2 + 1 = 3
3 + 3 = 6
```

Rückgabewerte: `0, 1, 3, 6`

Endrekursiv

```
return summeBis(n - 1, summe + n);
```

Vor jedem Aufruf wird gerechnet:

```
0 + 3 = 3
3 + 2 = 5
5 + 1 = 6
```

Rückgabewerte: `6, 6, 6, 6`

Ein `return` allein genügt nicht. Bei Endrekursion wird nach dem rekursiven Aufruf nur noch dessen Ergebnis unverändert zurückgegeben.

Endrekursion und Iteration

Endrekursiver Schritt

```
return summeBis(n - 1,
                summe + n);
```

Der nächste Aufruf erhält den
neuen Zustand als Parameter.

Dieselben Zustandsänderungen

```
int summeBisIterativ(int n) {
    int summe = 0;
    while (n > 0) {
        summe = summe + n;
        n = n - 1;
    }
    return summe;
}
```

Java garantiert keine Optimierung von Endrekursion zu einer Schleife.
Die Schleife benötigt für diese Berechnung keinen wachsenden Aufrufstack.

Rekursion oder Schleife?

Frage	Typische Orientierung
Wird nur ein Zustand fortgeschrieben?	Eine Schleife ist oft gut verständlich.
Entstehen kleinere Aufgaben derselben Art?	Rekursion kann den Aufbau der Aufgabe zeigen.
Sind sehr viele Schritte nötig?	Stackbedarf beachten, auch bei Endrekursion.
Wird der Code endrekursiv leichter lesbar?	Zusätzliche Hilfsparameter können mehr Erklärung benötigen.

Rekursion macht die berechenbaren Aufgaben nicht mächtiger. Sie ist eine andere Art, Abläufe zu beschreiben. Auch rekursiver Code kann Zahlenüberläufe erzeugen.

Gemeinsam: Aufrufe und Rückgaben verfolgen

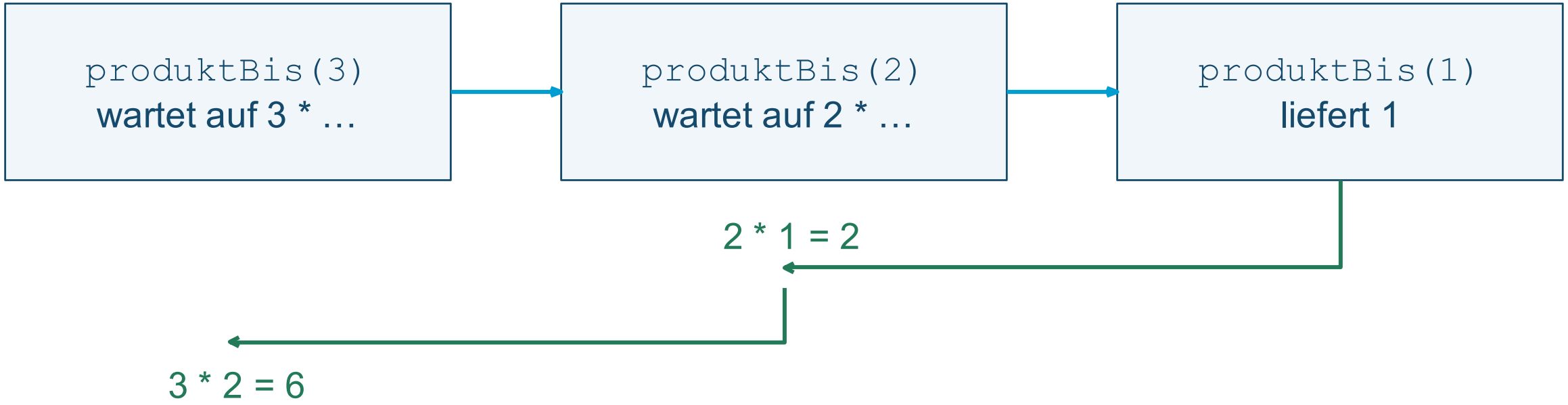
```
int produktBis(int n) {
    if (n == 1) {
        return 1;
    }
    return n * produktBis(n - 1);
}
```

Betrachten Sie `produktBis(3)`.

1. Welche Aufrufe entstehen?
2. Welcher Aufruf endet zuerst?
3. Welche Werte gehen zurück?
4. Ist die Methode endrekursiv?

Zeichnen Sie die Stackframes und markieren Sie den jeweils aktiven Aufruf.
Voraussetzung für dieses Beispiel: n ≥ 1.

Auflösung: `produktBis(3)`



Nicht endrekursiv: Nach der Rückkehr muss jeder wartende Aufruf noch multiplizieren.

Dokumentation zum Nachschlagen

Methodendeklarationen, Parameter und Überladen

Methodenaufrufe und Auswahl der passenden Methode

Die Rückgabeanweisung und das Ende eines Aufrufs

Javadoc-Kommentare und die verfügbaren Tags

JVM-Stack und Stackframes

Compact Source Files in Java 25

Lernziele für die anschließende Übung

Eigene Methoden entwerfen

Aufträge in Teilaufgaben zerlegen, Parameter und Rückgabewerte festlegen.

Methoden korrekt verwenden

Seiteneffekte unterscheiden, Überladungen aufrufen und Doc-Comments schreiben.

Aufrufe und Rekursion verstehen

Wertübergabe erklären, Stackframes verfolgen und Abbruchfälle prüfen.

Dabei verwenden wir weiterhin Java 25 oder höher und Compact Files.